PCCCS495 – Term II Project Report

Course Name: PCCCS495 – Advanced Programming (OOP) Lab

Project Title: Smart Email Filtering System using Strategy Pattern

Student Name: Arunima Ghosh

Instructor Name: Soumadip Biswas

Semester: Spring 2026

---

1. Problem Statement

Modern email systems receive a large number of messages daily, including personal, work-related, promotional, and spam emails. Manually organizing these emails is inefficient and error-prone. Important messages may be missed due to clutter, and spam emails may reduce productivity.

This project aims to design and implement a Smart Email Filtering System that automatically categorizes incoming emails into predefined folders such as Inbox, Work, and Spam. The system uses object-oriented programming principles and the Strategy Design Pattern to allow flexible and scalable classification rules.

2. System Design

2.1 Architecture Overview

The system follows a layered architecture:

UI Layer: Handles user input and display (Java Swing GUI)

Management Layer: Controls classification logic (EmailManager)

Strategy Layer: Implements filtering strategies (Filter Strategy interface and its implementations)

Model Layer: Represents data (Email class)

2.2 Major Classes

Email: Stores sender, subject, and content

Email Manager: Manages classification and storage

Filter Strategy (Interface): Defines filtering behavior

Spam Filter Strategy, Work Filter Strategy, Important Filter Strategy: Concrete implementations

Smart Email Filter UI: GUI interface

2.3 Package Structure

src/ → Java source code

docs/ → UML diagrams

report/ → Final report PDF

2.4 Interaction Flow

1. User enters email details in UI
2. UI sends data to EmailManager
3. EmailManager applies strategies
4. Email is categorized into a folder
5. Result is displayed on UI

3. OOP Concepts Demonstrated

3.1 Abstraction

Achieved using Filter Strategy interface

Hides implementation details of classification

3.2 Inheritance

Different filter strategies inherit from Filter Strategy

3.3 Polymorphism

Email Manager uses different strategies dynamically

3.4 Encapsulation

Email class encapsulates data fields

3.5 Exception Handling

Input validation and error handling using try-catch

3.6 Collections

Emails stored using ArrayList

4. Implementation Highlights

4.1 Strategy Pattern Implementation

The Strategy Pattern allows flexible classification logic.

Example: Spam Filter Strategy checks for keywords like “lottery”, “offer”

Work Filter Strategy checks for keywords like “meeting”, “project”

4.2 GUI Design

- Built using Java Swing
- Includes input fields and output area
- Provides buttons for classify, clear, and show folders

4.3 Dynamic Classification

- Emails are analyzed based on content and sender
- Assigned to appropriate folder automatically

5. Testing and Error Handling

Test Cases:

- Empty input → shows warning
- Spam keywords → moves to spam
- Work-related keywords → moves to work folder

Edge Cases:

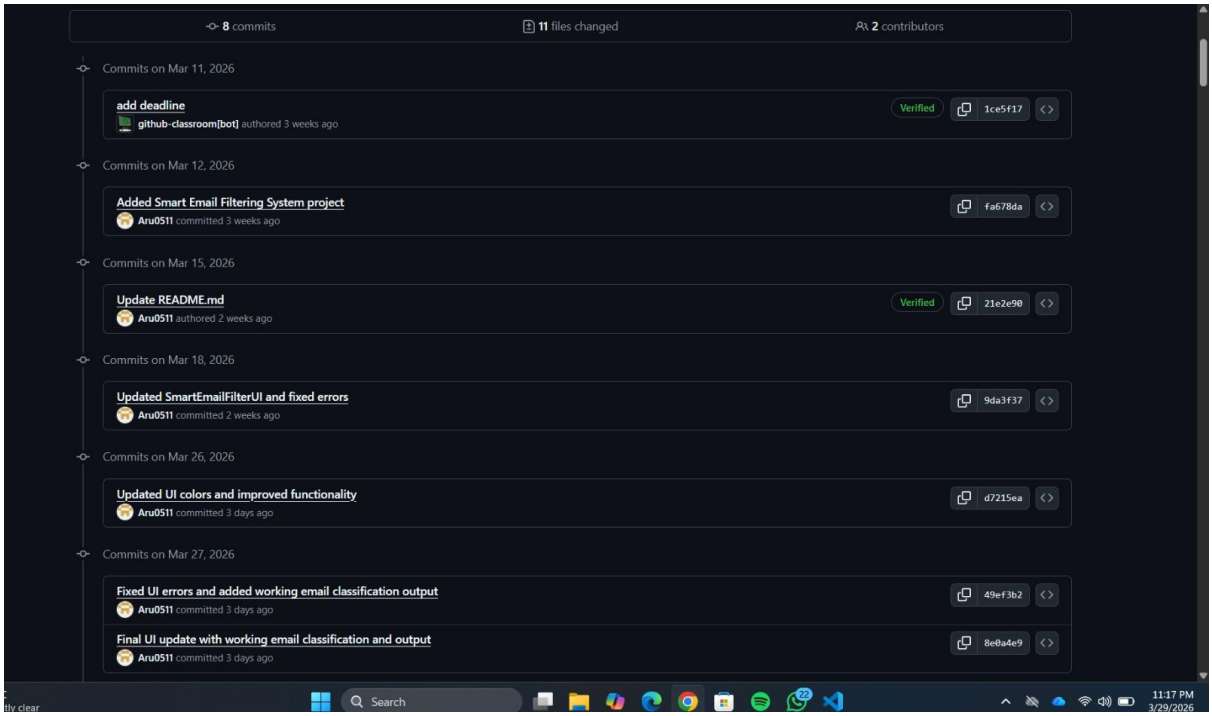
- Missing subject or sender
- Mixed keyword emails
- Failure Handling
- Invalid inputs handled with dialog messages

6. Git Workflow

- Repository created via GitHub Classroom
- Multiple commits made during development
- Changes tracked using Git

Example commits:

- Initial setup
- UI development
- Strategy implementation
- Final integration



7. Conclusion and Future Scope

The Smart Email Filtering System successfully demonstrates the use of OOP principles and design patterns. It provides an efficient way to categorize emails automatically.

Future Improvements

- Integration with real email APIs
- Machine learning-based classification
- User-defined rules

- Database storage

8.Appendix

Appendix A: Initial Project Proposal

The proposed project titled “Smart Email Filtering System using Strategy Pattern” aims to design an automated system that categorizes incoming emails into different folders such as Inbox, Work, and Spam.

The system is built using Object-Oriented Programming concepts and applies the Strategy Design Pattern to allow flexible and extensible email filtering rules.

The goal of the project is to:

1. Reduce manual effort in email organization
2. Automatically classify emails based on content and sender
3. Provide a scalable design where new filtering rules can be easily added

The proposal focuses on implementing core OOP concepts like abstraction, inheritance, polymorphism, encapsulation, exception handling, and collections.

Appendix B: Proposal Approval Email

Student Name: Arunima Ghosh

Project Title: Smart Email Filtering System using Strategy Pattern

Instructor: Soumadip Biswas

Approval Status: APPROVED_MINOR

Score: 88

Summary of Feedback:

i. The project demonstrates a clear understanding of OOP concepts and design patterns.
ii. The use of the Strategy Pattern ensures modularity and extensibility.
iii. The system architecture properly identifies components like Email entity, FilterStrategy interface, and EmailManager.

Minor improvements were suggested regarding:

- Management layer responsibilities
- Storage mechanism clarification (in-memory collections)
- Email input simulation
- Rule configuration flexibility
-
- Handling conflicts between multiple filters

9.Proposal Approval Email (Approved Minor)

Project: Smart Email Filtering System using Strategy Pattern
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal presents a clear and relevant problem that is well-suited for an object-oriented solution. The explicit use of the Strategy Design Pattern is a significant strength, demonstrating a solid understanding of how to achieve extensibility and modularity in a practical scenario. The architecture clearly identifies key components (Email entity, FilterStrategy interface, EmailManager context) and aligns well with the stated OOP concepts (Abstraction, Polymorphism, Encapsulation, Inheritance, Exception Handling, Collections). The emphasis on easy addition of new filtering rules is a direct benefit of the chosen pattern and addresses a common pain point. The inclusion of exception handling shows an awareness of building robust software.

Improvements:
1. **Clarify 'Management Layer':** While an `EmailManager` is described, the broader 'management layer' could be expanded upon. What other responsibilities fall into this layer beyond applying strategies and storing emails? E.g., user interaction, system configuration, more detailed folder management.
2. **Persistence Details:** The proposal states, 'All categorized emails are stored.' Specifying the intended storage mechanism (e.g., in-memory collections, simple file storage, or a basic database for a more advanced project) would add clarity regarding the scope of this feature.
3. **Email Source Simulation:** Clarify how 'incoming email messages' will be received for the project's scope. Will it be a simulated stream, reading from a predefined file, or basic user input?
4. **Rule Configuration/Management:** While the Strategy Pattern allows adding new *code-based* rules easily, consider briefly mentioning how these rules might be configured or dynamically managed from a user's perspective (e.g., basic config file, or just through code modification for a university project).
5. **Filter Prioritization/Conflict Resolution:** Briefly address how the system would handle emails that might match multiple filtering strategies (e.g., an email that is both 'Work' and 'Important'). Is there a defined order of application or a priority system?

10.UML DIAGRAMS:

System Architecture Diagram

```
User
  |
  v
SmartEmailFilterUI (UI Layer)
  |
  v
EmailManager (Logic Layer)
  |
  v
Email (Data Layer)
```

UML Class Diagram

```
SmartEmailFilterUI
------------------------
- senderField
- subjectField
- contentField
- outputArea
- manager
------------------------
+ classifyEmail()
+ showFolders()

         |
         v

EmailManager
------------------------
- inbox
- spam
- work
------------------------
+ classifyEmail()
+ showFolders()

         |
         v

Email
------------------------
- sender
- subject
- content
------------------------
+ constructor()
```

Use Case Diagram

```
         User
          |
 -------------------------
 |        |        |
Enter   Classify   View
Email   Email      Output
```

Sequence Diagram

```
User -> UI -> EmailManager -> Email
         |         |
         |         ---> classifyEmail()
         |
         <--- Result Display
```

Activity Diagram

```
Start
  |
Enter Email
  |
Click Classify
  |
Check Keywords
  |
+------+
|Spam? |
+------+
 | Yes -> Spam
 | No
 v
Work? -> Yes -> Work
 |
 No
 v
Inbox
 |
End
```

GUI Flow Diagram

```
User Input
   |
Click Button
   |
Validate Input
   |
Process Email
   |
Display Output
```